

APELLIDOS, NOMBRE:		DNI:	
I.E.S. DE REFERENCIA:	IES Aguadulce		
I.E.S. DONDE SE REALIZA EL EXAMEN:			

Selecciona tu profesor o profesora asignado:

☐ Salvador Romero Villegas (GRUPO B)	☐ Yonathan Martínez Castaño (GRUPO A)
---	--

INSTRUCCIONES

Este es un examen a realizar en PAPEL de contenido eminentemente práctico. Para la realización del examen:

- Debes tener el DNI visible durante la realización del examen y al finalizar el mismo.
- Para la realización del examen solo es necesario un bolígrafo negro o azul.
- Se prohíbe el uso de dispositivos electrónicos de cualquier tipo: teléfono móvil (el cual debe estar apagado en todo momento), reproductores multimedia o similares, auriculares u otro tipo de dispositivo sonoro, calculadoras, etc.
- El alumnado no puede llevar material adicional de ningún tipo.
- Durante la realización del examen el alumnado podrá disponer de tantas hojas adicionales como estime oportuno. Las hojas adicionales deberán graparse al examen y en ellas deberá constar el nombre y apellidos. Si las hojas adicionales no se desean adjuntar, deberán destruirse por el profesorado que vigila el examen.

Descripción del examen:

- Este examen comprende las unidades 1 (RA 1, 2 y 3), 2 (RA 6), 3 (RA 4), 4 (RA 5 parcialmente), 5 (RA 5 parcialmente), 6 (RA 7) y 7 (RAs 8 y 9), cada ejercicio está asociado a unos RAs diferentes.
- El examen está compuesto de 7 BLOQUES (A, B, C, D, E, F y G):
 - A) Cuatrimestre 1 - Ejercicios 1a y 1b: Uso de formularios, funciones y organización del código – Unidad 1
 - B) Cuatrimestre 1 - Ejercicio 2: Uso de base de datos – Unidad 2
 - C) Cuatrimestre 1 - Ejercicio 3: Uso de sesiones – Unidad 3
 - D) Cuatrimestre 2 – Cuestionario Práctico Unidad 4: aplicación del patrón MVC, uso básico composer y espacios de nombres PSR-4, uso básico clases y SMARTY.
 - E) Cuatrimestre 2 – Cuestionario Práctico Unidad 5: aplicación del patrón MVC usando Laravel (rutas, modelo Eloquent, controlador y vistas Blade)
 - F) Cuatrimestre 2 – Cuestionario Práctico Unidad 6: creación básica de un API REST en laravel (rutas, controlador y modelo), uso básico de API REST desde aplicación cliente con Guzzle
 - G) Cuatrimestre 2 – Cuestionario Práctico Unidad 7: creación básica de aplicación AJAX con JAXON-PHP/JAXON-JS.
- Las preguntas de los cuestionarios del segundo cuatrimestre incorrectas restan ¼.
- Los ejercicios a realizar dependen del resultado del examen de febrero: **¿Superaste el examen de febrero?**
 - **SI superado:** Realizar cuestionarios del cuatrimestre 2 (D, E, F y G) e intentar los ejercicios no superados en febrero según correspondencia.
 - **NO superado:** Realizar cuestionarios del cuatrimestre 2 (D, E, F y G) y todos los ejercicios de cuatrimestre 1.
- Correspondencia con examen de febrero:
 - Ejercicio 1 de Febrero → corresponde con → Bloque A) Cuatrimestre 1 - Ejercicios 1a y 1b.
 - Ejercicio 2 de Febrero → corresponde con → Bloque B) Cuatrimestre 1 - Ejercicio 2.
 - Ejercicio 3 de Febrero → corresponde con → Bloque C) Cuatrimestre 1 - Ejercicio 3.
- La nota final de las pruebas presenciales se calculará siguiendo la siguiente ponderación:
 - Nota pruebas presenciales: A*10,64% + B*20,03% + C*15,67% + D*8% + E*22% + F*18% + G*6%
 - **MUY IMPORTANTE:** la superación del examen de junio es obligatoria independientemente de si hay que realizarlo entero o solo una parte (por haber superado el examen de febrero). Si no se supera el examen de junio de forma independiente (teniendo en cuenta solo los ejercicios a realizar) no se aplicará la ponderación anterior.

CALIFICACIÓN

Mediante esta prueba se evalúan los resultados de aprendizaje y criterios de evaluación del módulo que se detallan a continuación:

Resultados de aprendizaje	Criterios de evaluación
RA1 (Primer cuatrimestre, Ejercicio 1)	f)
RA2 (Primer cuatrimestre, Ejercicio 1)	c), e), g)
RA3 (Primer cuatrimestre, Ejercicio 1)	a), b), c), d), e), f)
RA4 (Primer cuatrimestre, Ejercicio 3)	b), e), f)
RA6 (Primer cuatrimestre, Ejercicio 2)	b), c), d), e), f)
RA5 (Segundo cuatrimestre, tests Unidad 4 y Unidad 5)	c), d), e), f), g)
RA7 (Segundo cuatrimestre, tests Unidad 6)	d), e), f), g)
RA8 (Segundo cuatrimestre, tests Unidad 7)	f)
RA9 (Segundo cuatrimestre, tests Unidad 7)	g)

EXAMEN

BLOQUE A) Primer Trimestre - Ejercicios 1a y 1b - RA 1, 2 y 3

FRAGMENTO A1

```
$diasValidos = ['L', 'M', 'X', 'J', 'V', 'S', 'D'];  
$eventos = [];  
$diasSeleccionados=validarDatosRecibidos($diasValidos);  
if ($diasSeleccionados!==false) {  
    $eventos = eventos($diasSeleccionados);  
}
```

FRAGMENTO A2

```
<form method="POST">  
    <p>Haz clic aquí para seleccionar uno o más días de la semana:</p>  
    <?php  
    $diasSemana = [  
        'L' => 'Lunes', 'M' => 'Martes', 'X' => 'Miércoles', 'J' => 'Jueves',  
        'V' => 'Viernes', 'S' => 'Sábado', 'D' => 'Domingo'  
    ];  
    foreach ($diasSemana as $k => $n) {  
        echo "<label><input type='checkbox' name='dias[]' value='$k'> $n</label><br>";  
    }  
    ?>  
    <br>  
    <button type="submit">Enviar</button>  
</form>
```

FRAGMENTO A3

```
define("EVENTOS", [  
    'L' => ['opera', 'teatro'],  
    'X' => ['ballet', 'observación astronómica'],  
    'V' => ['cata gastronómica'],  
    'S' => ['museo de arte', 'cine'],  
    'D' => ['exposición científica']  
]);
```

Ejercicio 1a

Dado los fragmentos A1 y A2 escribe el código de la función `validarDatosRecibidos` la cual retornará los días recibidos vía POST solo si existen y además están dentro de los días válidos pasados por parámetro. En caso de que no se reciban datos vía POST o que los días recibidos no sean válidos se retornará `false`. Funciones que puedes utilizar:

- `is_array($x)`: permite verificar si la variable facilitada es un array
- `in_array($y, $x)`: permite verificar si \$y es un valor almacenado en el array \$x

Ejercicio 1b

Dados los fragmentos A1 y A2 crea la función `eventos` usada en el fragmento A1 de forma que genere un array con los eventos de los días pasados por parámetro. Por ejemplo, si `$diasSeleccionados` contiene `['L', 'V']` el array a retornar debería ser `['opera', 'teatro', 'cata gastronómica']`. Los eventos a usar estarán definidos en la constante `EVENTOS` declarada en el fragmento A3.

Importante: el código debe comprobar que la clave exista antes acceder a su valor en el array `EVENTOS`.

Bloque B) Primer Trimestre - Ejercicio 2 - RA 6

FRAGMENTO B4

```
CREATE TABLE coordenadas (  
    x INTEGER not null check (x>=0 AND x<16),  
    y INTEGER not null check (y>=0 AND y<16),  
    PRIMARY KEY (x,y)  
);  
INSERT INTO coordenadas (x,y) VALUES (2,5);  
INSERT INTO coordenadas (x,y) VALUES (1,9);
```

FRAGMENTO B5

```
$pdo = conectarBD();  
  
$x = isset($_GET['x']) ? intval($_GET['x']) : null;  
$y = isset($_GET['y']) ? intval($_GET['y']) : null;  
  
if ($x === null || $y === null || $x < 0 || $x > 15 || $y < 0 || $y > 15) {  
    echo "Coordenadas no válidas. Deben estar entre 0 y 15."  
    exit;  
}  
  
if (!existenCoordenadas($pdo,$x,$y)) {  
    $filasInsertadas=insertarCoordenadas($pdo,$x,$y)  
    if ($filasInsertadas===false)  
        echo "No se pudo insertar el registro";  
    else  
        echo "Filas insertadas=$filasInsertadas<BR>";  
}
```

Ejercicio 2

En el fragmento B4 se crea una tabla en la base de datos con coordenadas cartesianas (x,y) donde cada coordenada puede estar en el rango [0-15] (ambos inclusive). Dando por sentado que en el fragmento B5 se usará dicha tabla y que la variable \$pdo almacena una conexión PDO válida, crea la función insertarCoordenadas usada en el fragmento B5 para insertar datos en dicha tabla usando **PDO**, **consultas paramétricas**, **try/catch** y retornando un dato acorde al código donde se emplea.

Bloque C) Primer trimestre - Ejercicio 3 - RA 4

FRAGMENTO C6

```
<?php
session_start();

$numero = false;
if (isset($_GET['numero']) && is_numeric($_GET['numero']))
    $numero = $_GET['numero'];
$suma = calcular($numero);

?>
<h2>Introduce un número</h2>
<form method="GET">
    <input type="text" name="numero">
    <input type="submit" value="Enviar">
</form>
<p>La suma de los números actuales almacenados en la sesión es $suma ?></p>
```

Ejercicio 3

De forma acorde al código del fragmento C6 debes implementar la función `calcular` la cual almacenará un máximo de 5 números en la sesión (key `numeros`). Dicha función siempre retornará la suma de los números almacenados en la sesión (si no hay números almacenados o no existe el key `numeros` la suma será obviamente cero). Al almacenar un nuevo número, si hay más de 5 números almacenados se borra el más antiguo. Fíjate que la función puede recibir un número entero o `false`.

Funciones que puedes utilizar:

- `array_sum($x)`: calcular la suma del array `$x` pasado por parámetro.
- `$y=array_shift($x)`: eliminar el primer elemento el array `$x` y retornarlo en `$y`.

PLANTILLA RESPUESTAS CUESTIONARIOS

Importante: todas las preguntas del cuestionario se responderán en esta plantilla.

D1	D2	D3	D4	D5	D6	D7	D8
E1	E2	E3	E4	E5	E6	E7	E8
F1	F2	F3	F4	F5	F6	F7	F8
G1	G2	G3	G4	G5	G6	G7	G8

BLOQUE D) Segundo cuatrimestre - Unidad 4 - RA 4

FRAGMENTO D1

```
{
    "require": {
        "smarty/smarty": "^4.4.1"
    },
    "autoload": {
        "psr-4": { "EX\\": "src/" }
    }
}
```

FRAGMENTO D2

```
require __DIR__.'/vendor/autoload.php';

use EX\ctxls\extra\Operaciones;

$pdo=require __DIR__.'/conectarbd.php';

$smarty=new Smarty;
$smarty->setTemplateDir(__DIR__.'/res/templates');
$smarty->setCompileDir(__DIR__.'/res/compiled_templates');
$smarty->setCacheDir(__DIR__.'/res/smarty_cache');

switch ($_GET['op']??'')
{
    case 'suma':
        Operaciones::sumar($smarty);
        break;
    case 'restar':
        Operaciones::restar($smarty);
        break;
    default:
        Test::form($smarty);
        break;
}
```

FRAGMENTO D3

```
//...
use EX\mdl\Calculadora;

class Operaciones
{
    public static function sumar(\Smarty $smarty) { /*...*/}

    public static function restar(\Smarty $smarty)
    {
        $a=filter_input(INPUT_POST,'a');
        $b=filter_input(INPUT_POST,'b');
        if ($a && $b && is_numeric($a) && is_numeric($b))
        {
            $c=new Calculadora($a,$b);
            $smarty->assign('resultado',$c->diff());
        }
        $smarty->display('resultado.tpl')
    }
}
```

BLOQUE D) Segundo cuatrimestre - Cuestionario Unidad 4

D1. En el FRAGMENTO D1, ¿qué hay que cambiar para cambiar el nombre del espacio de nombres configurado?

- A) Una vez establecido, no se puede cambiar el espacio de nombres principal, solo añadir secundarios.
- B) Línea que comienza por "psr" donde pone "src".
- C) No hay ningún espacio de nombres, hay que añadir uno usando `classmap`.
- D) Línea que comienza por "psr" donde pone "EX".

D2. Dado el contenido del FRAGMENTO D1 y suponiendo que está guardado en el archivo correspondiente en disco, ¿qué comando hay que ejecutar para disponer de Smarty?

- A) `php artisan install`
- B) `composer require`
- C) `php artisan dump-autoload`
- D) `composer install`

D3. Dado el contenido del FRAGMENTO D1 y del FRAGMENTO D2, piensa en el directorio donde se almacena el archivo `Operaciones.php`, ¿qué tenemos que añadir en el FRAGMENTO D2 para hacer uso de la clase `Test` si dicha clase está en un archivo ubicado en el directorio inmediatamente superior (padre) al del archivo `Operaciones.php`?

- A) `use src\EX\ctxls\Test;`
- B) `use EX\ctxls\Test;`
- C) `use EX\ctxls\extra\Test;`
- D) `use EX\Test;`

D4. Dado el contenido del FRAGMENTO D1 y del FRAGMENTO D2, ¿cómo sería el espacio de nombres a añadir en el archivo `Test.php` teniendo en cuenta la premisa de que dicho archivo está ubicado en el directorio inmediatamente superior al del archivo `Operaciones.php`?

- A) `namespace EX\ctxls;`
- B) `namespace EX\ctxls\extra;`
- C) `namespace EX\ctxls\extra\Test;`
- D) `namespace EX\ctxls\Test;`

D5. Dado el código del FRAGMENTO D3, ¿cómo sería el constructor de la clase `Calculadora`?

- A) `public function Calculadora($a, $b) { ... resto del código ... }`
- B) `public Calculadora(int $a = 0, int $b = 0) {};`
- C) `public function __construct($a, $b) { ... resto del código ... }`
- D) `public __construct(int $a, int $b) { ... resto del código ... }`

D6. Dado el código del FRAGMENTO D3, imagina que no se ejecuta el código que hay dentro del `if` porque no se cumplen las condiciones, ¿cuál de las siguientes opciones previene un error en la plantilla Smarty usada?

- A) `{ if(isset($resultado)) } {$resultado} {/if}`
- B) `@isset($resultado)`
`{ $resultado }`
`@endif`
- C) `@if (isset($resultado))`
`{ {$resultado} }`
`@endif`
- D) `{ if(isset($resultado)) } @resultado {/if}`

D7. ¿Cuál entre los FRAGMENTOS D2 y D3 representa un controlador y por qué?

- A) El FRAGMENTO D3 porque es el encargado de modificar y manipular los datos del modelo.
- B) El FRAGMENTO D2 porque es el que tiene la lógica de negocio, se encarga de decidir que hacer, es como el director de orquesta.
- C) Ninguno de los fragmentos corresponde con un controlador. Un ejemplo de controlador sería la clase `Calculadora`.
- D) El FRAGMENTO D3 porque es el encargado de realizar cada operación concreta usando el modelo a conveniencia y pasando los datos correspondientes a la vista.

D8. Dentro del FRAGMENTO D3, concretamente dentro del método `restar`, ¿cómo sería el código del método `diff`?

- A) `public function diff() { return $this->a - $this->b; }`
- B) `public static function diff() { return static::$a - static::$b; }`
- C) `public function diff($a,$b) { return $a - $b; }`
- D) `public static function diff($a, $b) { return $a - $b; }`

Bloque E) Segundo cuatrimestre - Unidad 5 - RA 4

FRAGMENTO E4

```
Route::get('/mascota/votar/', [MCT::class, 'votarMascota'])->name('votar');
Route::get('/mascota/{mascota}/convertir/', [MCT::class, 'convertirMascota'])->name('convertir');
Route::post('/mascota/{mascota}/borrar', [MCT::class, 'borrarMascota'])->name('borrarmascota');
```

FRAGMENTO E5

```
use HasFactory;
protected $fillable = ['nombre', 'publica', 'megustas', 'user_id','descripcion'];
public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}
```

FRAGMENTO E6

```
public function editarMascota(int $masc) {
    return view('formmascota');
}
public function nuevaMascota(Request $request)
{
    $data=$request->validate([
        'nombre' => 'required|string|max:50',
        'descripcion' => 'required|string|max:250',
        'publica' => 'required|string|in:Si,No',
    ]);

    $mascota = new Mascota();
    $mascota->nombre = $data['nombre'];
    $mascota->publica = $data['publica'];
    $mascota->descripcion = $data['descripcion'];
    $mascota->save();
}
```



```
return view('mascotacreada', ['m'=>$mascota]);
}
```

Bloque E) Segundo cuatrimestre – Cuestionario unidad 5

E1. Dado el FRAGMENTO E4, ¿qué se ejecutaría en función de la ruta?

- A) votar, convertir y borrar mascota
- B) votarMascota, convertirMascota y borrarMascota
- C) votar, convertir y borrar
- D) get y post

E2. Para hacer referencia a la primera ruta del FRAGMENTO E4 en una plantilla blade, ¿qué pondríamos?

- A) Votar
- B) Votar
- C) Votar
- D) Votar

E3. El FRAGMENTO E5 es el código interno de una clase del modelo almacenada en el archivo Mascota.php, ¿cuál de las siguientes opciones sobre dicha clase NO es correcta?

- A) La clase debe extender la clase Migration.
- B) La clase debe estar en el espacio de nombres App\Models.
- C) La clase debe llamarse Mascota.
- D) La clase será usada por Eloquent.

E4. El FRAGMENTO E5 es el código interno de una clase del modelo almacenada en el archivo Mascota.php, ¿cuál de las siguientes opciones nos permite indicar el nombre de la tabla de la base de datos donde se almacenarán los datos?

- A) private \$dbtable = 'mascota';
- B) protected \$table = 'mascotas';
- C) const \$table = Migration::table('mascotas');
- D) private function setTable() { return 'mascotas'; }

E5. El FRAGMENTO E5 declara una variable llamada \$fillable, ¿cuál es el propósito de dicha variable?

- A) \$fillable es un array que contiene los campos de la tabla que no se pueden rellenar de forma masiva.
- B) \$fillable es un array que contiene los campos de la tabla que no pueden estar vacíos.
- C) \$fillable es un array que contiene los campos de la tabla que se pueden rellenar de forma masiva.
- D) \$fillable es un array que contiene los campos de la tabla que no se rellenan de forma automática (tales como timestamps o id autogenerado).

E6. La vista formmascota usada en el FRAGMENTO E6, ¿qué tipo de vista es?

- A) Es una vista de tipo HTML, dado que no tiene parámetros.
- B) Es una vista de tipo PHP, dado que Blade y Smarty son compatibles con PHP.
- C) Es una vista de tipo Smarty como en todos los proyectos Laravel.
- D) Es una vista de tipo Blade, dado que es el motor de plantillas por defecto en este caso.

E7. En la vista mascotacreada usada en el FRAGMENTO E6, ¿cómo podría mostrarse el nombre de la mascota recién guardada?

- A) <h1>{{ \$mascota['nombre'] }}</h1>
- B) <h1>{{ \$mascota->nombre }}</h1>
- C) <h1>{{ \$m->nombre }}</h1>
- D) <h1>{{ \$m['nombre'] }}</h1>

E8. En primer método del FRAGMENTO E6 vas a usar los datos de la mascota con el identificador pasado por parámetro y los vas a pasar a la vista formmascota, ¿cómo sería el código para buscar la mascota con identificador recibido por parámetro si la clase del modelo se llama Mascota?

- A) \$mascota=Mascota::find(\$masc);
- B) \$mascota=Mascota::select('id_mascota', \$masc);
- C) \$mascota=Mascota::only('id_mascota', \$masc);
- D) \$mascota=Mascota::where('id_mascota', \$masc);

Importante: las respuestas del cuestionario se responderán en la plantilla

Bloque F) Segundo cuatrimestre - Unidad 6 - RA 7

FRAGMENTO F7

```
Route::get('/mascotas', [ApiController::class, 'listarMascotas']);
Route::post('/nuevamascota', [ApiController::class, 'crearMascota']);
Route::put('/mascota/{mascota}', [ApiController::class, 'cambiarDescripcionMascota'])->
whereNumber('mascota');
Route::delete('/mascota/{mascota}', [ApiController::class, 'borrarMascota'])->
whereNumber('mascota');
```

FRAGMENTO F8

```
function cambiarDescripcionMascota(Request $request, Mascota $mascota): JsonResponse
{
    if (!$request->isJson()) {
        return response()->json('ERROR', 403);
    }

    $v = Validator::make($request->json()->all(),
        [
            'descripcion' => 'required|string|max:250'
        ]
    );

    if ($v->fails()) {
        return response()->json('Datos incorrectos', 400);
    }

    $datos = $v->validated();

    $mascota->descripcion = $datos['descripcion'];
    $mascota->save();

    return response()->json('Mascota modificada correctamente.', 200);
}
```

FRAGMENTO F9

```
$client = new Client($options);
$url='mascota/' . urlencode($id_mascota);
$response = $gclient->put($url, ['json' => $datos]);
switch($response->getStatusCode())
{
    case 400:
        $data = json_decode($response->getBody());
        $datosVista['mensaje'] = $data;
        break;
    case 200:
        $data = json_decode($response->getBody());
        $datosVista['mensaje'] = $data;
        break;
    default:
        $datosVista['mensaje'] = "Respuesta no esperada: ".$response->getStatusCode();
        break;
}
```

Bloque F) Segundo cuatrimestre – Cuestionario Unidad 6

F1. Dado el FRAGMENTO F7, imagina que se puede registrar la fecha en la que una mascota fallece, ¿qué método HTTP sería más conveniente en un API REST para que un usuario pudiese registrar la fecha de fallecimiento de una mascota?

- A) Route::get
- B) Route::put
- C) Route::post
- D) Route::delete

F2. Dado el FRAGMENTO F7, ¿en qué archivo encontraríamos esas líneas de código?

- A) routes/web.php
- B) app/http/controllers/api/APIController.php
- C) routes/api.php
- D) app/http/controllers/api/routes.php

F3. ¿Por qué no se usa `whereNumber` en la segunda ruta del FRAGMENTO F7?

- A) Porque no es necesario, ya que el parámetro de ruta que tiene es un formulario y no un número.
- B) Porque se entiende que en este caso es el método controlador quien verifica a través de Request si el parámetro de ruta es numérico o del tipo que sea.
- C) Porque no se puede usar `whereNumber` en una ruta de tipo POST.
- D) Porque la ruta no tiene un parámetro de ruta.

F4. Dado el FRAGMENTO F7 y el FRAGMENTO F8, ¿qué ocurriría si se cambia el segundo parámetro del método `cambiarDescripcionMascota` por `int $dato`?

- A) Habría que eliminar `whereNumber` de la ruta correspondiente, dado que ya no es necesario, y hacer las verificaciones oportunas dentro del método.
- B) No tendría sentido dicho cambio, ya que el número se podría recoger a través de `Request $request`. Si se deja como está tienes la ventaja de que obtienes la instancia de `Mascota` directamente.
- C) Dentro del método habría que buscar la mascota correspondiente a dicho número, y, si existe, hacer la operación oportuna.
- D) No se podría hacer el cambio, ya que el segundo parámetro no puede ser un número.

F5. Dado el FRAGMENTO F8, en el último `return`, queremos cambiarlo y devolver dos valores (`$v1` y `$v2`) en una respuesta en formato JSON, ¿cómo sería la forma más lógica de hacerlo?

- A) `return response()->json(json_encode([$v1,$v2]),200);`
- B) `return response()->json([$v1,$v2],200);`
- C) `return response()->json(json_encode([$v1,$v2]),200);`
- D) `return response()->json([$v1,$v2],200);`

F6. Dado el FRAGMENTO F8 y el FRAGMENTO F9, solo en una de las siguientes situaciones **NO SE EJECUTA** la rama "default" del switch del FRAGMENTO F9, ¿sabrías decir cuál?

- A) Cuando la descripción se envía al servicio web como datos de un formulario normal y corriente.
- B) Cuando la descripción tiene 300 caracteres.
- C) Cuando el id de la mascota no corresponde a una mascota real.
- D) Cuando se produce un error interno en nuestra aplicación y se retorna el código 505.

F7. Atendiendo al código de FRAGMENTO F7 y FRAGMENTO F8, ¿cómo podría ser el contenido de la variable `$datos` usada en la tercera línea del FRAGMENTO F9?

- A) `$datos = ['descripcion'=>'Nueva descripción','mascota'=>2];`
- B) `$datos = json_encode(['descripcion'=>'Nueva descripción']);`
- C) `$datos = ['descripcion'=>'Nueva descripción'];`
- D) `$datos = json_encode(['descripcion'=>'Nueva descripción','mascota'=>2]);`

F8. Atendiendo al código del FRAGMENTO F7, si en el código del FRAGMENTO F9 tuviera que enviar los datos como formulario normal y corriente, ¿qué tengo que poner al usar el cliente HTTP Guzzle?

- A) `$gclient->put($url, ['form_params' => json_decode($datos)])`
- B) `$gclient->put($url, ['form_params' => $datos])`
- C) `$gclient->post($url, ['form_params' => $datos])`
- D) `$gclient->post($url, ['form_params' => json_decode($datos)])`

Bloque G) Segundo cuatrimestre - Unidad 7 - RAs 8 y 9

FRAGMENTO G10

```
$jaxon->register(Jaxon::CALLABLE_FUNCTION, 'borrarLibro');
$jaxon->register(Jaxon::CALLABLE_FUNCTION, 'registrarLibro');
```

FRAGMENTO G11

```
function borrarLibro($isbn)
{
    $response = new Response();

    if (!ctype_digit($isbn) && strlen($isbn) != 13) {
        $error="El ISBN solo puede contener 13 dígitos numéricos.";
    }
    elseif (!$pdo=DB::getConn())
    {
        $error="No se puede conectar con la base de datos";
    }
    else {
        $c=Libro::borrarPorISBN($pdo,$isbn);
        $mensaje="Libros eliminados $c (ISBN=$isbn)";
    }
    if (isset($error))
        $response->alert($error)
    elseif (isset($mensaje))
        $response->assign('mensaje','innerHTML',$mensaje);
    return $response;
}
```

FRAGMENTO G12

```
<form id="eliminarLibro" onSubmit="return false;">
    ISBN del libro:<input id="isbn" data-id="isbn" type="text" name="isbn"><BR>
    <input type="button" value="Borrar"
        onclick="<?=rq()->call('borrarLibro',pm()->input('isbn'))?>">
</form>
```

Bloque G – Segundo cuatrimestre – Cuestionario Unidad 7

G1. Dado el fragmento G11, ¿qué habría que añadir en el fragmento G12 para que se muestre el mensaje en caso de que se elimine el libro?

- A) <P data-id="mensaje"></H1>
- B) <P name="mensaje"></H1>
- C) <H1 id="mensaje"></H1>
- D) <P>{\$mensaje}</H1>

G2. Dado el fragmento G12, ¿cuál de los siguientes códigos JavaScript sería equivalente al usado en onclick?

- A) onclick="<?=jaxon_call('borrarLibro',jaxon.\$('isbn').value);?>"
- B) onclick="<?=jaxon_borrarLibro(pm()->input('isbn'))?>"
- C) onclick="jaxon_borrarLibro(document.getElementById('isbn').value);"
- D) onclick="let v=document.getElementById('mascota').value; jaxon.borrarLibro(v);"

G3. Dado el fragmento G11, la línea que comienza por \$response->assign, ¿qué hace?

- A) Establece el valor de la variable \$mensaje como contenido del elemento HTML con name="mensaje".
- B) Establece el valor de la variable \$mensaje como contenido del elemento HTML con data-id="mensaje".
- C) Ninguna de las otras opciones es correcta.
- D) Establece el valor de la variable \$mensaje como contenido del elemento HTML con id="mensaje".

- G4.** Dado el fragmento G10 y el fragmento G11, ¿tiene sentido declarar `registrarLibro` como `private function registrarLibro (...)`?
- A) No tiene sentido, ya que la función no se va a definir dentro de una clase.
 - B) No tiene sentido. Si se pone la función es privada y no es accesible vía AJAX.
 - C) No, ya que están en el mismo archivo donde se registran. Aunque se ponga `private`, se puede registrar igualmente.
 - D) Solo tiene sentido si la función se va usar después de que el usuario se haya autenticado.
- G5.** Dado el fragmento G10, ¿cuál es el propósito del método `register`?
- A) Registrar una función JavaScript para que pueda ser llamada vía AJAX usando JAXON desde el servidor.
 - B) Registrar una función PHP para que pueda ser llamada desde el backend usando JAXON-PHP.
 - C) Registrar una función PHP para que pueda ser llamada vía AJAX desde el cliente web.
 - D) Registrar una función JavaScript para que pueda ser llamada desde el servidor web para obtener información del cliente web.
- G6.** Dado el fragmento G11 y G12, ¿qué ventajas tiene sobre una aplicación web tradicional?
- A) Se sobrecarga menos el servidor, dado que la eliminación del libro se hace en el cliente web.
 - B) La actualización de la interfaz de usuario es más segura, dado que se envía la mínima información al servidor (solo el ISBN del libro) y no la página completa.
 - C) Se sobrecarga menos el cliente, dado que todo el código se ejecuta en el servidor y no en el cliente.
 - D) La actualización de la interfaz de usuario es más rápida ya que solo se actualiza el contenido necesario. En cualquier caso, puede complementar la forma de funcionamiento más tradicional.
- G7.** Dado el fragmento G12, ¿qué ocurriría si los datos del formulario se envían vía POST tradicional en vez de usar AJAX?
- A) En ese caso los datos se reciben por JAXON-PHP como un array asociativo, y habría que usar `filter_input` o equivalente dentro de la función `borrarLibro` para acceder a los datos.
 - B) En ese caso los datos se reciben por JAXON-PHP como un array asociativo y el mismo se encarga de transformarlos a lo que necesita la función registrada.
 - C) Tendríamos que preparar un script PHP aparte para recibir los datos del formulario, procesarlos y devolver una respuesta al cliente que contuviera el HTML completo que deseamos que se muestre al cliente.
 - D) Usar envío de formularios tradicional no es compatible con JAXON-PHP ni JAXON-JS, por lo que generaría error PHP.
- G8.** En el fragmento G12 se desencadena una petición AJAX en vez de una petición HTTP POST tradicional, imagina que en el fragmento G11 responde con el texto `return "ERROR: revisa ISBN"`; en vez de retornar una instancia de la clase `Response`, ¿qué ocurriría en el cliente web?
- A) La cadena retornada es en sí misma una cadena en formato JSON, por lo que JAXON-JS lo interpretará como un texto que hay que mostrar en el navegador en la salida por defecto. La salida por defecto es la consola de log del navegador.
 - B) Lo más probable es que provoque un fallo en el cliente web, modifique la estructura DOM del documento HTML de forma inadecuada y deje de funcionar.
 - C) JAXON-PHP va a transformar de forma automática el texto en cuestión en un mensaje JSON que acepta JAXON-JS, con lo que JAXON-JS lo interpretará adecuadamente.
 - D) La página que está visualizando el usuario simplemente no se modifica, al no recibir una respuesta JSON apropiadamente formateada JAXON-JS no puede interpretar el contenido y no se altera el DOM del documento.